

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo E (azul).
Profesor: Isabel Pita.

Nombre del alumno:

1. Dadas las siguientes funciones de coste, indica su orden de complejidad (representado por la función más sencilla) y ordena los órdenes de complejidad obtenidos utilizando $\subset$.

- a) $f_1(n) = 5n^2 + 3n + 1$
- b) $f_2(n) = n + \log n + 3$
- c) $f_3(n) = n^5 + 3n^3 + 2$
- e) $f_4(n) = \log_2(n + 3) + 5$
- d) $f_5(n) = \log_3 n$

Respuesta:

- a) $f_1(n) = 5n^2 + 3n + 1 \in \mathcal{O}(n^2)$
- b) $f_2(n) = n + \log n + 3 \in \mathcal{O}(n)$
- c) $f_3(n) = n^5 + 3n^3 + 2 \in \mathcal{O}(n^5)$
- d) $f_4(n) = \log_2(n + 3) + 5 \in \mathcal{O}(\log n)$
- e) $f_5(n) = \log_3 n \in \mathcal{O}(\log n)$

$$\mathcal{O}(\log n) \subset \mathcal{O}(n) \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^5)$$

2. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n >> k;
for (int i = 0; i < n; ++i){
    for (int j = 0; j < k; ++j) ++x;
    for (int j = k; j < n; ++j) --x;
}
```

Respuesta:

El bucle externo da n vueltas, siendo n el valor de entrada. Para calcular el coste del bucle nos falta comprobar si todas las ejecuciones del cuerpo del bucle tienen el mismo coste y en caso afirmativo cual es el coste de una ejecución del cuerpo del bucle.

El cuerpo del bucle externo consta de dos bucles `for` ejecutados secuencialmente. El primero de ellos da k vueltas, siendo k el valor de entrada y el coste de las instrucciones del bucle es constante porque se trata de un incremento de una variable que podemos suponer de un tipo básico. Por lo tanto el coste del primer bucle interno es $\mathcal{O}(k)$. El segundo bucle interno da $n - k$ vueltas y el coste de las instrucciones del bucle es constante debido a que se trata en este caso de un decremento de una variable que suponemos de un tipo básico. Como los dos bucles se ejecutan de forma secuencial, el coste es el máximo de ambos, por lo tanto será $\mathcal{O}(\max(n, k))$, siendo n y k los valores de entrada.

Por lo tanto el coste del bucle externo es $n * \max(n, k) = \max(n^2, n * k)$. Si suponemos $k < n$ el coste está en $\mathcal{O}(n^2)$, en otro caso está en $\mathcal{O}(n * k)$.

3. Escribe un predicado para expresar que los valores de un vector v , entre las posiciones i y j son positivos.

Respuesta:

$$\text{positivos}(v, i, j) \equiv \forall k : i \leq k \leq j : v[k] > 0$$

4. Escribe un predicado para expresar que el número de valores pares de un vector v entre las posiciones i y j es mayor que un cierto valor x .

Respuesta:

$$pares(v, i, j, x) \equiv (\#k : i \leq k \leq j : v[k] \% 2 == 0) > x$$

5. Especifica un algoritmo que dado un vector de números enteros, cuya longitud es mayor que cero, calcule la longitud de la secuencia más larga de números positivos tal que más de la mitad de los valores de la secuencia sean números pares.

Respuesta:

En este caso nos piden la especificación completa por lo que hay que dar la precondition y la postcondition.

$$P : \{v.size() > 0\}$$

`secPositivos(vector<int> v) dev int l`

$$Q : \{l == \max p, q : 0 \leq p \leq q < v.size() \wedge pares(v, p, q, (q - p)/2) : q - p\}$$

6. Dada la especificación:

$$P : \{v.size() \geq 0\}$$

`resolver(vector<int> v) dev vector<int> sol`

$$Q : \{\forall x : 0 \leq x < sol.size() : sol[x] = \sum k : x \leq k < v.size() : v[k]\}$$

Y dada la siguiente implementación:

```
std::vector<int> resolver(std::vector<int> const& v) {
    std::vector<int> sol(v.size());
    sol[v.size()-1] = v[v.size()-1];
    for (i = v.size()-1; i > 0; --i) {
        sol[i-1] = v[i-1] + sol[i];
    }
    return sol;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo:

Respuesta:

$$Q : \{\forall x : i \leq x < sol.size() : sol[x] = \sum k : x \leq k < v.size() : v[k] \wedge 0 \leq i < v.size()\}$$

7. Dado un vector de números enteros cuyos elementos están ordenados en orden decreciente, indica un algoritmo *óptimo* que permita obtener el máximo del vector.

Respuesta:

El algoritmo devolverá el primer elemento del vector.

8. Define una función recursiva que resuelva el problema de contar el número de veces que aparece el dígito 1 en un número. Observad que se pide la función recursiva.

Respuesta:

$$digito1(n) = \begin{cases} 0 & \text{if } n < 10 \wedge n \neq 1 (\text{caso base}) \\ 1 & \text{if } n < 10 \wedge n == 1 (\text{caso base}) \\ digito1(n/10) & \text{if } n \geq 10 \wedge n \% 10 \neq 1 \\ digito1(n/10) + 1 & \text{if } n \geq 10 \wedge n \% 10 == 1 \end{cases}$$

9. Indica una recurrencia que permita calcular el coste en el caso peor del siguiente algoritmo, haz el desplegado y obtén el orden de complejidad del algoritmo.

```
bool f ( std::vector<int> const& v, int ini, int fin, int x){
    if (ini == fin) return true; // Vector vacio
    else {
        int m = (ini + fin - 1) / 2;
```

```

    if (v[m] > v[ini]) return f(v,ini,m,x);
    else if (v[m] <= v[ini] && x < v[m]) return f(v,m,fin,x);
    else return f(v, ini, m+1,x) || f(v,m,fin,x);
}
}

```

Respuesta:

En el caso peor se realizan dos llamadas recursivas (ver la última instrucción del condicional) cada una con la mitad de los elementos del vector. El coste de cada llamada recursiva es constante, porque solo se realizan operaciones aritméticas y lógicas, instrucciones de asignación y accesos a vector.

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ 2T(n/2) + c_1 & \text{if } ini < fin \end{cases}$$

El desplegado y el coste final se encuentran en el cuaderno de problemas de complejidad de algoritmos recursivos del campus.

10. Indica cuál es la recurrencia que permite calcular el coste del algoritmo quicksort en el caso peor y en el caso medio. Indica el orden de complejidad del algoritmo en ambos casos.

Respuesta:

En el caso peor, el pivote se sitúa siempre en un extremo del vector, dando lugar a una llamada recursiva con el vector vacío y otra llamada con $n - 1$ elementos, siendo n el número de elementos del vector de partida. La recurrencia es:

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ T(n-1) + n & \text{if } ini < fin \end{cases}$$

cuyo coste está en $\mathcal{O}(n^2)$.

En el caso medio el pivote se sitúa aproximadamente en el centro del vector, dando lugar a dos llamadas recursivas cada una con aproximadamente la mitad de los elementos del vector. La recurrencia es:

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ 2T(n/2) + n & \text{if } ini < fin \end{cases}$$

cuyo coste está en $\mathcal{O}(n \log n)$.